INDEX

Sr. No.	Title Of Assignment
1	Write a program to implement an address book with options given below: a) Create address book. b) View address book. c) Insert a record. d) Delete a record e) Modify a record. f) Exit
2	Process control system calls: The demonstration of FORK and WAIT system calls along with zombie and orphan states. Implement the C program in which main program accepts the integers to be sorted. Main program uses the FORK system call to create a new process called a child process. Parent process sorts the integers using sorting algorithm and waits for child process using WAIT system call to sort the integers using any sorting algorithm. Demonstrate zombie and orphan states.
3	Thread synchronization using counting semaphores and mutual exclusion using mutex. Application to demonstrate: producer-consumer problem with counting semaphores and mutex.
4	Implement the deadlock-free solution to Dining Philosophers problem to illustrate the problem of deadlock and/or starvation that can occur when many synchronized threads are competing for limited resources.
5	Implement the C program for Deadlock Avoidance Algorithm: Bankers Algorithm.
6	FIFOs: Full duplex communication between two independent processes. First process accepts sentences and writes on one pipe to be read by second process and second process counts number of characters, number of words and number of lines in accepted sentences, writes this output in a text file and writes the contents of the file on second pipe to be read by first process and displays on standard output.
7	Full duplex communication between parent and child processes. Parent process writes a pathname of a file (the contents of the file are desired) on one pipe to be read by child process and child process writes the contents of the file on second pipe to be read by parent process and displays on standard output.
8	Implement the C program for Disk Scheduling Algorithms: SSTF, SCAN, C-Look considering the initial head position moving away from the spindle.

ASSIGNMENTNO. 1

SHELL PROGRAMMING

AIM : Write a program to implement an address book with options given below:
b) Create address book. b) View address book. c) Insert a record.
d) Delete a record. e) Modify a record. f) Exit.

OBJECTIVES : To study

1. Basic Shell commands
2. Shell script

THEORY :

Shell Script: Normally shells are interactive. It means shell accept command from you (via keyboard) and execute them. But if you use command one by one (sequence of 'n' number of commands), then you can store this sequence of command to text file and tell the shell to execute this text file instead of entering the commands. This is known as shell script. Shell Script is series of command written in plain text file. This manual is meant as a brief introduction to features found in Bash.

Exit Status:

By default in Linux if particular command/shell script is executed, it return two type of values which is used to see whether command or shell script executed is successful or not.

(1) If return value is zero (0), command is successful.

(2) If return value is nonzero, command is not successful or some sort of error executing command/shell script.

This value is known as Exit Status. But how to find out exit status of command or shell script? Simple, to determine this exit Status you can use $? Special variable of shell.

For e.g. (This example assumes that file1 does not exist on your hard drive)

$ rm file1

It will show error as follows

rm: cannot remove `file1': No such file or directory

and after that if you give command

$ echo $?

it will print nonzero value to indicate error.

Variables in Shell

To process our data/information, data must be kept in computers RAM memory. RAM

memory is divided into small locations, and each location had unique number called memory location/address, which is used to hold our data. Programmer can give a unique name to this memory location/address called memory variable or variable (Its a named storage location that may take different values, but only one at a time).

In Linux (Shell), there are two types of variable:

(1) System variables - Created and maintained by Linux itself. This type of variable defined in CAPITAL LETTERS.

(2) User defined variables (UDV) - Created and maintained by user. This type of variable defined in lower letters.

You can see system variables by giving command like $ set, some of the important System variables are:

System Variable	Meaning
BASH=#!/bin/bash	Our shell name
BASH_VERSION=1.14.7(1)	Our shell version name
HOME=/home/vivek	Our home directory
LOGNAME=students	students Our logging name
OSTYPE=Linux	Our Os type
PATH=/usr/bin:/sbin:/bin:/usr/sbin	Our path settings
PWD=/home/students/Common	Our current working directory
SHELL=/bin/bash	Our shell name

You can print any of the above variables contains as follows:

$ echo $HOME

test command or [expr]

test command or [expr] is used to see if an expression is true, and if it is true it return zero(0), otherwise returns nonzero for false.

Syntax:

test expression OR [expression]

Example:

Following script determine whether given argument number is positive.

```
if test $1 -gt 0
then
echo "$1 number is positive"
fi
```

test or [expr] works with

1. Integer (Number without decimal point)
2. File types
3. Character strings

For Mathematics, use following operator in Shell Script

Mathematical Operator in Shell Script	Meaning	Normal Arithmetical/ Mathematical Statements	But in Shell	
			For test statement with if command	For [expr] statement with if command
-eq	is equal to	5 == 6	if test 5 -eq 6	if [5 -eq 6]
-ne	is not equal to	5 != 6	if test 5 -ne 6	if [5 -ne 6]
-lt	is less than	5 < 6	if test 5 -lt 6	if [5 -lt 6]
-le	is less than or equal to	5 <= 6	if test 5 -le 6	if [5 -le 6]
-gt	is greater than	5 > 6	if test 5 -gt 6	if [5 -gt 6]

-ge	is greater than or equal to	5 >= 6	if test 5 -ge 6	if [5 -ge 6]

NOTE: == is equal, != is not equal.

For string Comparisons use

Operator	Meaning
string1 = string2	string1 is equal to string2
string1 != string2	string1 is NOT equal to string2
string1	string1 is NOT NULL or not defined
-n string1	string1 is NOT NULL and does exist
-z string1	string1 is NULL and does exist

Shell also test for file and directory types

	Meaning
-s file	Non empty file
-f file	Is File exist or normal file and not a directory
-d dir	Is Directory exist and not a file
-w file	Is writeable file
-r file	Is read-only file
-x file	Is file is executable

Logical Operators:

Logical operators are used to combine two or more condition at a time

Operator	Meaning
! expression	Logical NOT
expression1 -a expression2	Logical AND

expression1 -o expression2	Logical OR

if condition

if condition which is used for decision making in shell script, If given condition is true then command1 is executed.

Syntax:

```
if condition
then
        command1 if condition is true or if exit status
        of condition is 0 (zero)
        …
        …
fi
```

Condition is defined as:

"Condition is nothing but comparison between two values."

For compression you can use test or [expr] statements or even exist status can be also used.

Loops in Shell Scripts

Bash supports:

1) for loop

2) while loop

while :

The syntax of the while is:

```
while test-commands
do
commands
done
```

Execute *commands* as long as *test-commands* has an exit status of zero.

for :

The syntax of the for is:

```
for variable in list
do
commands
done
```

Each white space-separated word in list is assinged to variable in turn and commands executed until list is exhausted.

The case Statement

The case statement is good alternative to Multilevel if-then-else-fi statement. It enable you to match several values against one variable. Its easier to read and write. Syntax: case $variable-name in

```
pattern 1)   command
             ...
             ..
                command
                ;;
Pattern 2)   command
             ...
             ..
                command
                ;;
Pattern N)   command
             ...
             ..
                command
                ;;
*)           command
             ...
             ..
                command
                ;;
```

The $variable-name is compared against the patterns until a match is found. The shell then executes all the statements up to the two semicolons that are next to each other. The default is *) and its executed if no match is found.

Program 1 : `#!/bin/bash`

```
# Define the data file
ADDRESS_BOOK_FILE="address_book.txt"

# Function to add a contact
add_contact() {
    echo "Enter Name:"
    read name
    echo "Enter Phone Number:"
    read phone
    echo "Enter Email Address:"
    read email
    echo "$name;$phone;$email" >> "$ADDRESS_BOOK_FILE"
    echo "Contact added."
}

# Function to view all contacts
view_contacts() {
    if [ -s "$ADDRESS_BOOK_FILE" ]; then
        echo "--- Address Book ---"
        cat "$ADDRESS_BOOK_FILE" | while IFS=';' read -r name phone email; do
            echo "Name: $name"
            echo "Phone: $phone"
            echo "Email: $email"
            echo "-----------------------------------------------"
         }done
    else
        echo "Address book is empty."
    fi

# Function to search for a contact
search_contact() {
    echo "Enter name or part of name to search:"
    read search_term
    if [ -s "$ADDRESS_BOOK_FILE" ]; then
        grep -i "$search_term" "$ADDRESS_BOOK_FILE" | while IFS=';' read -r
name phone email; do
            echo "Name: $name"
            echo "Phone: $phone"
            echo "Email: $email"
            echo "-----------------------------------------------"
         }done
    else
        echo "Address book is empty."
    fi
# Function to delete a contact
delete_contact() {
    echo "Enter name of contact to delete:"
    read delete_name
    if [ -s "$ADDRESS_BOOK_FILE" ]; then
        temp_file=$(mktemp)
        grep -iv "^$delete_name;" "$ADDRESS_BOOK_FILE" > "$temp_file"
        mv "$temp_file" "$ADDRESS_BOOK_FILE"
        echo "Contact deleted (if found)."
```

```
else
  echo “Address book is empty”.
fi
 }

 # Main menu loop
 while true; do
     echo ""
     echo "Address Book Menu:"
     echo "1. Add Contact"
     echo "2. View Contacts"
     echo "3. Search Contact"
     echo "4. Delete Contact"
     echo "5. Exit"
     echo "Enter your choice:"
     read choice

     case $choice in
         1) add_contact ;;
         2) view_contacts ;;
         3) search_contact ;;
         4) delete_contact ;;
         5) exit 0 ;;
         *) echo "Invalid choice. Please try again." ;;
     esac
 done
```

Conclusion: Thus in shell script we can write series of commands and execute as a single program and we implemented address book.

Answer the following questions.

1. What are different types of shell & differentiate them.
2. Explain Exit status of a command.
3. Explain a) User define variables.
4. System variables.
5. What is man command?
6. Explain test command.
7. Explain how shell program get executed.
8. Explain syntax of if-else, for, while, case.
9. Explain use of functions in shell and show it practically.
10. Write a menu driven shell script to execute different commands with options.
11. Execute same assignment in another shell.